Market Making with Deep Reinforcement Learning from Limit Order Books

Hong Guo
Shenzhen International Graduate School
Tsinghua University
Shenzhen, China
guoh20@mails.tsinghua.edu.cn

Jianwu Lin *
Shenzhen International Graduate School
Tsinghua University
Shenzhen, China
Lin.Jianwu@sz.tsinghua.edu.cn

Fanlin Huang
Microsoft
Beijing, China
fanlinghuang@microsoft.com

arXiv:2305.15821v1 [q-fin.CP] 25 May 2023

***Abstract*—Market making (MM) is an important research topic in quantitative finance, the agent needs to continuously optimize ask and bid quotes to provide liquidity and make profits. The limit order book (LOB) contains information on all active limit orders, which is an essential basis for decision-making. The modeling of evolving, high-dimensional and low signal-to-noise ratio LOB data is a critical challenge. Traditional MM strategy relied on strong assumptions such as price process, order arrival process *etc.* Previous reinforcement learning (RL) works handcrafted market features, which is insufficient to represent the market. This paper proposes a RL agent for market making with LOB data. We leverage a neural network with convolutional filters and attention mechanism (Attn-LOB) for feature extraction from LOB. We design a new continuous action space and a hybrid reward function for the MM task. Finally, we conduct comprehensive experiments on latency and interpretability, showing that our agent has good applicability.**

***Index Terms*—market making, reinforcement learning, quantitative finance, high frenquency trading**

I. Introduction

Market making (MM) is a salient problem in the quantitative finance domain. Market makers usually quote limit orders on both ask and bid sides to capture spreads and make profits. Market makers, who provide liquidity and immediacy to the market, play a crucial role in the price discovery process.

While market makers are faced with three main costs: adverse information costs (45%), order processing costs (45%), and inventory holding costs (10%) [1]. Adverse information costs are the cost paid by uninformed traders to informed traders. Order processing costs refer to commission fees, such as stamp duty, transfer fees, *etc.* Inventory holding costs are due to inventory, which could suffer losses when the value of the asset fluctuates.

More than half of the markets in today's financial world use an electronic limit order book (LOB) to facilitate trade [2]. The LOB is a set of all active orders submitted to the Exchange. Cao et al. [3] found that the LOB is informative and contributes approximately 22% to price discovery.

The challenge is that the state space of a LOB is huge, which makes it very difficult to investigate conditional dependencies. Therefore, a key modeling task is to find a way to simplify the evolving, high-dimensional state space, while retaining LOB's important features [4]. In an electronic matching market, to develop an autonomous market making agent has become a practical problem.

This paper presents an adaptive agent for market making under a simulated environment. [1] A CNN-Attention based network (Attn-LOB) is proposed as a function approximator to extract features from LOB. There are three main challenges, the first is the feature extraction from the noisy LOB data, the second is to define a suitable action space for market making, and the last is to reward the agent properly to the right decision, which is a challenging task even for a human trader.

A. Related Work

For market making, Demsetz [5] first proposed a theory, he pointed out the bid-ask spread is the compensation that markets provide for the immediacy of transactions. Then, Garman et al. [6] proposed an inventory based model for market making. Afterwards, the model developed cases from a single market maker to multiple market makers [7], [8]. Many theoretical market making models are developed in the context of stochastic dynamic programming. A widely used model was proposed by Avellaneda and Stoikov [9], who calibrated the optimal quotes by considering the probability with which his quotes will be executed as a function of their distance from the mid-price, this work followed Ho and Stoll [8], [10]. Avellaneda-Stoikov (AS) model assumes that the price is continuous, which is not true because of the minimum price tick. Guilbaud et al. [11] constructed a strategy for discrete quotes. The main limitation of these models is that specific properties of the underlying processes (price process and order arrival process) have to be assumed in order to obtain a closed-form characterization of strategies [12].

The ever rising speed and decreasing costs of computational power and networks have led to the emergence of huge databases that record all transactions and order book movements up to milliseconds [13]. Many econophysics literature [6], [14]–[16] studied the microstructure of LOBs. In recent years, with the popularity of data-driven methods, a lot of data-driven methods [17]–[20] tried to extract features from noisy

* Corresponding author.

[1]The source code is available at https://github.com/imTurkey/Market-Making-with-Deep-Reinforcement-Learning-from-Limit-Order-Books

financial data. And machine learning methods [21]–[25] were applied to short-term price forecasting. Also, deep learning methods such as Convolutional Neural Network (CNN) [26], Recurrent Neural Network (RNN) [27] and Attention [28], [29] were leveraged to predict price movement and discover patterns in the high-frequency trading (HFT) domain.

The above methods were mainly used to predict signals. Many RL methods were used for trading [30]–[34], optimal execution [35] and portfolio management [36]–[38].

Many work applied RL to market making, Abernethy [39] is the first to leverage online learning to solve market making problem. Then, Lim and Gorse [40] and Spooner et al. [41] applied non-deep RL for simulated and real-world historic data respectively, [40] used inventory and remaining time to describe the state, [41] took handcrafted LOB features such as spread, price movement, order imbalance into account, their action spaces are both discrete. Zhong [42] handcrafted features to characterize market volatility and mid-price movement, but their action space allowed the agent to exit the market. Sadighian [43] first leveraged Deep Reinforcement Learning (DRL) in cryptocurrency market making, he used LOB, TFI (Trade Flow Imbalance) and OFI (Order Flow Imbalance) to describe the market state, he leveraged a MLP as the function approximator, which could have limited the ability of LOB description. Gašperov and Kostanjčar [44] used a gradient boosting model to predict realized price range and a long short-term memory (LSTM) to predict the trend, and the signals were used to train a DRL agent. Some latest researches [45], [46] leveraged Deep Recurrent Q-Networks (DRQN) and Dueling Double Deep Q Network (D3QN) respectively, however, neither of them considered the characteristics of LOB while designing deep networks.

These work have either used simulated market data, handcrafted features or used a simple MLP for the feature extraction from LOBs, which could have restricted the market-making ability of reinforcement learning agents.

B. Contributions

The main contribution of this paper is the design of an adaptive agent for market making with reinforcement learning from limit order book. In the financial domain, many scenario-specific metrics such as latency, inventory and Sharpe are investigated, and a simulated backtesting paradigm of RL for market making is established. In contrast to previous MM models [9], [11] which rely on strong assumptions, we use a model-free approach which need not to explicitly model the environment. Compared with the previous RL agents [39]–[41], [44], [47] which handcrafted market representation, we extract abundant stationary and dynamic market features from LOB data, which is proved effective for market making. Our main contributions can be summarized below:

- We propose an effective framework to train an RL agent with automatic feature extraction and pre-training from LOB data for market making problems, which is a novel challenge in the HFT domain.
- We propose a CNN-Attention based network (Attn-LOB) for automatic feature extraction from LOB data, we pre-trained it in a mid-price direction prediction task, Attn-LOB is proved effective for establishing the representation of the market.
- We propose a novel continuous action space close to the financial practice for training the RL agent, we compared it to a previous discrete action space which limits the agent to make decisions.
- We study several reward functions designed for capturing spreads and controlling inventory in theory and experiments. We design a hybrid reward function for market making and show their effectiveness.
- We conduct an interpretability experiment to explain our agent, we find that the agent can pay attention to the relevant market changes and learn the basic skills of market making, which showed great practical potential.

II. Preliminaries

A. Limit Order Book

There are two types of orders: Market Orders and Limit Orders. A market order is an order to buy or sell at the market's current best available price, which typically ensures an execution but does not guarantee a specified price. It is executed against limit orders starting with the best price. In a less-liquid market, a market order may spill over to further price levels if there is not enough volume at the current level of the order book. A limit order is an order to buy or sell with a restriction on the maximum price to be paid or the minimum price to be received (the "limit price"), it does not assure execution. When a limit order arrives at the Exchange, it joins the back of the order queue at its quoted price level and will be filled on a first-come first-served rule. The collection of all active limit orders is a limit order book (LOB).

The state space of a limit order book is inherently high-dimensional, posing a significant challenge to modeling efforts. For instance, if there exist P price levels, the simplest aggregated state space would be $\mathbb{Z}^P$, which renders the modeling process complex. Furthermore, the limit order book is often characterized by noise, owing to the presence of numerous irrelevant signals, such as quoting and immediately canceling orders.

B. Market Making

A market maker (MMer) is a company or an individual that quotes both buy and sell limit orders in an asset to make a profit. An MMer's goal is to earn the difference between the quoted bid and ask prices. If both orders get executed, then the MMer will gain the quoted spread. In an ideal case, the inventory-based MMer seeks to get both orders executed to minimize the inventory risk. But there is a possible scenario that only one of these orders gets executed, which results in the MMer getting a non-zero inventory, thereby exposing itself to the inventory risk. When faced with inventory risk, the MMer typically skews its quotes in order to reduce its inventory position.

C. Reinforcement Learning

Reinforcement learning (RL) is a machine learning method used to solve sequential decision problems. In the trading domain, RL provides a way to directly output decisions using an end-to-end training process rather than a two-stage prediction and decision-making process. Since the decision targets are difficult to label, the supervised learning methods are difficult to implement, and RL approaches provide a new perspective to train agents through reward signals.

III. Method

A. Market Representation

1) Order Book State: The snapshot of an order book at time t can be simplified to a one-dimensional vector whose length is $4 * n$, where n represents the number of levels.

$$LOB_t = \left\{ P_{i,t}^{\text{ask}}, V_{i,t}^{\text{ask}}, P_{i,t}^{\text{bid}}, V_{i,t}^{\text{bid}} \right\}_{i=1}^{n} \tag{1}$$

Where $P_{i,t}$ denotes the price of level i at time t, $V_{i,t}$ denotes the total volume of level i at time t . We adopt a rolling window of length T to form the input, the shape of which is (T, 4*n). Handcrafted features are limited and insufficient to describe a high-frequency market. We pre-trained a deep neural network and applied this model as a function approximator during reinforcement learning, which will be explained in Section III-B.

2) Dynamic market state: The limit order book only contains stationary information. In this section, we design a new indicator and leverage two commonly used indicators to obtain the dynamic market states.

- **Order Strength Index** (OSI). To consider the dynamic impact of newly happened events on the market, we customize OSI to describe the relative strength between the bid and ask. The events can be divided into three categories: new market orders, new limit orders and order cancellations. The strength indexes in both volumes and numbers of each category are calculated by:

$$OSI_v = \frac{\sum V_{buy} - \sum V_{sell}}{\sum V_{buy} + \sum V_{sell}} \tag{2}$$

  where V denotes the volume, and we also calculate OSI based on the number of orders. OSI is calculated on the time window of 10s, 60s and 300s respectively to obtain 18 features.
- **Realized Volatility** (RV) is an assessment of variation for assets by analyzing its historical returns within a defined period, it can be calculated by:

$$RV = \sqrt{\sum_{t=1}^{n} \left(\log p_t - \log p_{t-1} \right)^2} \tag{3}$$

  where p_t denotes the price at time t. We calculate the RV for the past 5 minutes, 10 minutes, and 30 minutes respectively, and get 3 features.
- **Relative Strength Index** (RSI) [48] is a technical indicator used in momentum trading that measures the speed of a security's recent price changes to evaluate overvalued or undervalued conditions in the price of that security. It can be calculated by:

$$RSI = \frac{\sum_{t=1}^{n} Gain_t}{\sum_{t=1}^{n} Gain_t + \sum_{t=1}^{n} Loss_t} \tag{4}$$

  Where $Gain_t = max(0, p_t - p_{t-1})$, $Loss_t = max(0, p_{t-1} - p_t)$. Similarly, we calculate the RSI in 5 minutes, 10 minutes, and 30 minutes respectively, and obtain 3 features.

3) Agent State: Agent state contains inventory and time factor. **Inventory** is a critical internal state to control the inventory risk. **Remaining Time** is calculated by current time t divided by total time T.

B. Limit Order Book Modeling

We use a CNN-Attention based network to extract features from the LOB data, and we use pre-train [49] to improve its performance. The mid-price direction prediction is a widely studied problem and can be used as the pre-train task. The task is to classify the future price trend (up, down or stationary) in a certain horizon with limit order book of the past t timestamps.

1) Label Acquisition: We use the approach proposed in [23] to obtain labels, which are proved to be more robust:

$$y_t = \begin{cases} 1 & \text{if } l_t > \alpha \\ 0 & \text{if } -\alpha \le l_t \le \alpha \\ -1 & \text{if } l_t < -\alpha \end{cases} \tag{5}$$

where α is a threshold hyperparameter which can be set according to the market. And l_t can be calculated by

$$l_t = \frac{m_+(t) - m_-(t)}{m_-(t)} \tag{6}$$

$$m_\pm(t) = \frac{1}{k} \sum_{i=0}^{k} p_{t \mp i} \tag{7}$$

Where p_t is the mid-price and k denotes the predicted horizon,

2) Normalization: Since the value of an asset may fluctuate to a never before seen level, statistics of the price values can change significantly with time, rendering the price time series non-stationary. In order to transform the non-stationary input sequence into a stationary sequence, we adopt normalization described in [18]. Afterwards, we perform *z-norm* to the stationary price sequence and *max-norm* for the volume sequence.

3) Model: The model we use, Attn-LOB, is a deep neural network which is composed by convolutional networks and self-attention. The model architecture follows [19], [23]. They proposed a CNN-LSTM based network and we add a *multi-head self-attention* layer to model temporal dependencies. The network architecture is shown in Figure 1. The input shape is $T \times (4 * n)$. First, multiple convolutional filters are used to model the spatial dependencies. Then the output in shape $T \times hidde_dim$ are fed into an *Inception* module to establish multi-scale temporal dependencies. At last, a *multi-head self-attention* layer was used to aggregate temporal dependencies.

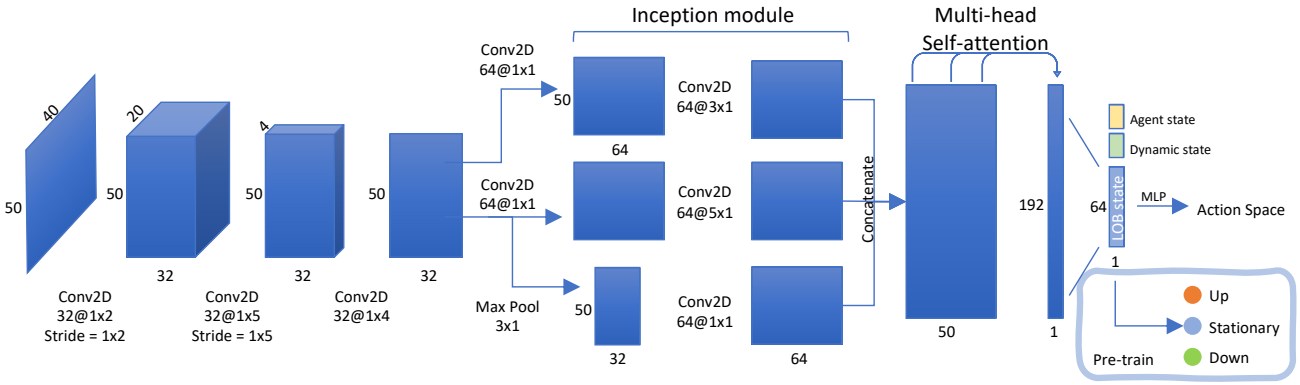


Fig. 1. The model architecture.

C. Action Space

A typical market making strategy is to submit limit orders on both buy and sell sides. The agent is restricted to a single buy and sell order and cannot exit the market, which is similar to [50]. Here we introduce two action spaces. The first is discrete and has been used many times in past work. The second is a continuous action space designed by us.

1) Discrete: The discrete action space A is similar to [41], [43] and consists of 8 possible actions. Action 0-7 is to quote a pair of orders with a particular spread and bias, and action 7 is to close position with market orders. The ask and bid prices will be calculated based on the price when orders are submitted. The volume of each order is the $minimum_trade_unit$, which is 100 in the China Stock Market.

2) Continuous: The continuous action space is more flexible than the discrete action space mentioned above. This type of action space is inspired by the AS strategy [9], which obtains optimal ask and bid quotes by calculating a reservation price p_r and a quotes spread, p_r can be seen a "true" price of the asset [8], and the agent symmetrically quotes orders around p_r.

Action A_1 and A_2 are between 0 and 1. A_1 controls the bias δ of reservation price p_r and mid-price p_m:

$$\delta = A_1 * max_bias \tag{8}$$

Where max_bias is a hyper-parameter to decide the maximum bias of reservation price and mid-price. Then reservation price p_r can be calculated by:

$$p_r = p_m - \text{Sign}(Inventory) * \delta \tag{9}$$

The bias direction of reservation price is controlled by inventory, because we want the agent to buy when its inventory is negative and sell when its inventory is positive.

Action A_2 controls the qouted spread:

$$spread = A_2 * max_spread \tag{10}$$

where max_bias and max_spread are hyper-parameters to decide the maximum price bias and the maximum quotes spread respectively. The final quoted ask and bid prices are:

$$p_{a,b} = p_r \pm spread/2 \tag{11}$$

To avoid unlimited buying and selling, we set a hyper-parameter ω , when the agent's position is higher than $\omega * minimum_trade_unit$, the agent will be prohibited from placing orders in that direction.

D. Reward Function

1) Dampened PnL: The reward function is the key to guiding reinforcement learning. A natural thinking is to use Profit and Loss (PnL) as the reward function, which refers to the difference of value of the agent in Δt:

$$\Delta PnL_t = value_t - value_{t-\Delta t} \tag{12}$$

Where value is calculated by the sum of inventory's value (calculated with current mid-price) and cash. Spooner et al. [41] has proved that only using PnL as the reward function will lead to the agent's speculative action. The agent tend to hold a large amount of inventory, which brings a great risk of drawdown when the trend market comes. To avoid this, they added an asymmetric punishment term to calculate the Dampened PnL (DP):

$$DP_t = \Delta PnL_t - max(0, \eta * \Delta PnL_t) \tag{13}$$

Where η is a hyper-parameter. DP will reduce the reward for profit from holding, but not the punishment for loss.

2) Trading PnL: Trading PnL (TP) rewards the agent only at transaction $X_t = (X_{t,p}, X_{t,v})$, where $X_{t,p}$ denotes the transaction price and $X_{t,v}$ denotes the transaction volume. TP is defined as:

$$TP_t = X_{t,v} * (p_{m,t} - X_{t,p}) \tag{14}$$

Where $p_{m,t}$ denotes the mid-price at time t. $X_{t,v}$ is positive for buying and negative for selling. It can be viewed as an advantage over the mid-price when trading. We can prove that $Trading\ PnL$ is equivalent to ΔPnL subtract $Holding\ PnL$, the latter is PnL caused by positions, defined by $Holding\ PnL = Inv_{t-\Delta t} * (p_{m,t} - p_{m,t-\Delta t})$. In short, $Trading\ PnL$ can reward the price advantage of trading rather than the profit or loss of inventory.

3) Inventory Punishment: The inventory punishment (IP) is an effective way to control inventory risk:

$$IP_t = \zeta * Inv_t^2 \tag{15}$$

Where ζ is a hyper-parameter. In order to make the inventory punishment quickly increase, we use L_2 norm to calculate IP.

4) Reward Function: Finally, we combine these rewards above to formulate a hybrid reward function as Equation (16). DP_t are used to punish loss from holding inventory, TP_t are used to reward advantageous price of trading, and IP_t are used to punish the high inventory position.

$$R_t = DP_t + TP_t - IP_t \tag{16}$$

E. Training Algorithm

We adopt two commonly used RL algorithms to train our agent: Dueling DQN is a value-based SOTA, and Proximal Policy Optimization (PPO) is a policy-based SOTA.

IV. Experiments

A. Dataset

The data we use are historical orders and trades on the Shenzhen Stock Exchange in November 2019, covering 21 trading days. Data are updated event-by-event and an event may occur due to anything from a change in price, volume or arrangement of orders. We select three stocks in different industries to study the applicability of proposed agent, which are *Ping An Bank Co.,Ltd.* (SZ.000001) in banking, *Wuliangye Yibin Co.,Ltd.* (SZ.000858) in liquor-making, and *Hikvision Technology Co.,Ltd.* (SZ.002415) in technology. We reconstruct their historical limit order books, which contain about 5,000,000 samples.

B. Pre-training

1) Experimental Setup: We conduct our experiments on Ubuntu 20.04 LTS. The machine consists of an Intel Xeon(R) E5-2650 v4 CPU and 48GB DRAM. The training data is the first half of the month (10 days), and the test data is the second half of the month (11 days), with 20% of the training data used for validation. We followed the setting of [25] and only took data between 10:00 and 11:30, 13:00 and 14:30, during which trading was considered stable. We labeled these data as described in Section III-B1. We set the predicted horizon k to 10 events, and the label threshold α=1e-5, and the windows length $T = 50$.

2) Baselines: FC-LOB is a multi-layer perception network. The number of neurons in the hidden layer is (1024, 256, 64, 3), and the activation function is leaky Relu except for Softmax in the last layer. Conv-LOB is a fully convolutional network that uses dilated convolution to accept longer sequences. The architecture is similar to [51]. DeepLOB is the network proposed by [23]. And Attn-LOB is our model described in Section III-B3.

TABLE I
Pre-train results.

	Precision	Recall	F1	Param	Input
FC-LOB	0.6315	0.5419	0.5660	256,064	4000×1
Conv-LOB	0.5851	0.5230	0.4984	172,320	1024×40
DeepLOB	**0.7856**	0.6699	0.7118	139,168	100×40
Attn-LOB	0.7663	**0.7019**	**0.7284**	176,320	50×40

3) Pre-train Results: We present the results of pre-training on one stock ($Ping\ An\ Bank\ Co., Ltd.$), and we also note that the model still performs well on stocks out of sample. As shown in Table I, Attn-LOB outperforms other methods on recall and F1 score, and we only use half the time length as input compared with DeepLOB. Conv-LOB does not achieve the desired result, suggesting that we may not need to input such a long time sequence.

C. RL Settings

1) Simulator: We set up a simulated trading environment with the historical limit order book. To ensure the reality of simulated transactions, the simulator executes the agent's order only when the real historical order arrives. When the agent's bid price is higher than the lowest market ask price, or the agent's ask price is lower than the highest market bid price, the order gets executed and then the agent's cash and inventory will be updated. The transaction cost is set to 0, and the agent can hold negative cash or inventory. If the absolute inventory exceeds $max_inventory = \omega * minimum_trade_unit$, the agent will be prohibited from quoting in that direction. If the agent chooses to close positions with market orders, the order will be executed at the counterparty's price. Since the volume of agent's quote is small, we ignore the impact of these orders on the market.

2) Experimental Setup: For the convenience of training and testing, we divide a trading day to some small episodes, each of which includes 2000 events, which is about 3-5 minutes. In each episode, the agent's value is initialized to 0, and at the end of each episode, the agent will close positions with market orders. The PnL of this episode will be the final value of the agent. We split the train and test data as described in Section IV-B1. In the training phase, the agent can go through the training data once or more, while in the test phase, the agent can only go through it once. We set hyperparameters $\omega = 10$, $T = 50$, $\eta = 0.5$, $\zeta = 0.01$.

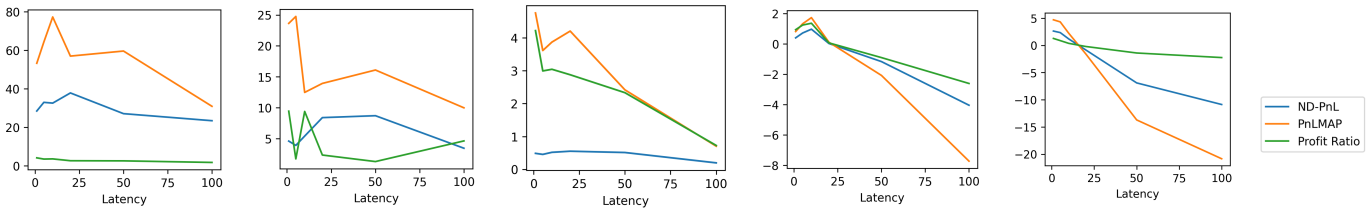
Dueling DQN performed better in the discrete action space, so we use it to train the agent in the discrete action space. And PPO is used to train the agent in the continuous action space. The Discrete Dueling DQN is called D-DQN, The Continuous PPO is called C-PPO. The hyperparameters used in continuous action space are $max_bias = 0.05$, $max_spread = 0.1$.

3) Compared Methods:

- **Inventory-based RL** [40] is an RL baseline, they used the agent's $inventory$ and $remaining_time$ to represent market state.
- **LOB-based RL** [42] is another RL baseline which hand-crafted market feature from limit order books. They used

TABLE II
OVERALL RESULTS.

	Ping An Bank Co.,Ltd.				Wuliangye Yibin Co.,ltd.				Hikvision Technology Co., Ltd.			
	ND-PnL ($\times 10^5$)	PnLMAP	PR ($\times 10^{-4}$)	Sharpe	ND-PnL ($\times 10^5$)	PnLMAP	PR ($\times 10^{-4}$)	Sharpe	ND-PnL ($\times 10^5$)	PnLMAP	PR ($\times 10^{-4}$)	Sharpe
C-PPO	9.3±0.7	117.2±3.8	5.0±0.1	12.3±0.8	19.8±1.8	630.6±85.5	2.8±0.5	2.2±0.7	16.0±3.3	313.6±25.9	3.8±0.6	7.1±0.5
D-DQN	7.0±1.7	8.6±2.2	3.5±0.2	1.3±0.7	11.0±3.7	28.4±12.8	0.9±0.1	-0.5±0.1	11.7±6.8	65.2±10.0	10.1±3.3	0.4±0.1
Inv-RL	0.3±0.1	24.7±4.2	4.3±0.9	1.3±0.3	3.8±0.4	70.2±23.0	0.7±0.1	-1.3±0.2	1.4±0.1	52.7±16.6	2.4±0.4	1.0±0.4
LOB-RL	1.1±0.5	1.3±0.5	2.8±1.4	0.2±0.3	1.8±1.1	6.8±5.0	0.2±0.1	-0.7±0.3	1.9±0.9	5.9±3.9	0.7±0.4	-0.2±0.4
AS	0.49	4.75	4.22	0.74	3.14	19.61	3.93	0.17	1.57	16.39	8.10	0.65
Random	0.39	0.81	0.93	-0.19	0.86	3.33	0.15	-0.81	2.76	6.73	1.75	0.27
Fixed_1	2.63	4.70	1.28	-0.01	-3.12	-10.72	-0.12	-4.88	1.43	3.71	0.24	-1.69
Fixed_2	0.97	2.03	9.97	0.21	7.66	26.57	2.36	0.55	4.49	10.55	6.38	1.02
Fixed_3	0.25	1.41	21.58	0.31	3.84	13.36	3.89	0.36	1.62	4.60	10.53	0.48



(a) C-PPO (b) D-DQN (c) AS (d) Random (e) Fixed

Fig. 2. Latency experiments.

$bidSpeed$, $askSpeed$, $avgmidChangeFrac$, $invSign$, $cumPnL$ to represent the market.

- **Avellaneda-Stoikov Strategy** [9] is a classical model commonly used in the market making. It calculates the optimal quote price with a stochastic control model. The reservation price and optimal spread are calculated by:

$$r(s,q,t) = s - q\gamma\sigma^2(T-t) \tag{17}$$

$$\delta^a + \delta^b = \gamma\sigma^2(T-t) + \frac{2}{\gamma}\ln\left(1+\frac{\gamma}{\kappa}\right) \tag{18}$$

where s is the current mid-price, q is the quantity of assets in inventory, σ is a volatility parameter, γ is a risk aversion parameter, κ is a liquidity parameter, T is the total time and t is the current time.

- **Random Quoting Strategy** randomly quotes ask and bid orders in the five levels of the limit order book.
- **Fixed Quoting Strategy** constantly quotes ask and bid orders in the fixed (1-3) level of the limit order book.

4) Metrics:

- **ND-PnL** [41] can rate how good our strategies are at capturing spreads. It is defined as PnL divided by the average spread in a period. It means how many spreads are captured by the agent on average.
- **PnLMAP** [44] is defined as PnL divided by the mean absolute position (MAP) in this period. It means the PnL in per unit of inventory and can measure the ability of the agent to profit against inventory risk.
- **Profit Ratio** (PR) is defined as the PnL divided by the total trading volume of the agent. This metric measures the agent's profitability against transaction costs.
- **Sharpe ratio** compares the return of an investment with its risk to evaluate the long-term profitability of a strategy.

D. Overall Results

As Table II shows, our RL agents achieve competitive performance especially in capturing spread and inventory controlling. C-PPO, in particular, beats all baselines, showing the advantage of the proposed continuous action space. For profit ratio, we notice that the smaller the quoting distance is, the lower the PR is, but the profitability increases (Fixed3 rendered the highest PR while the almost lowest ND-PnL and PnLMAP). The reason is that there are many useless transaction at the same price when quoting closely, so there is a trade-off. For Sharpe ratio, we can see only our C-PPO and AS strategy still remain positive among all the targets, showing our stability of profits. For capturing spreads, our C-PPO and D-DQN have a huge advantage. The best Fixed strategy also has a good performance (sometimes even defeats AS strategy), but it is not stable, whose optimal quoted distance differs from stocks (level 1 for *Ping An Bank Co.,Ltd.* and level 2 for the other). And their inventory levels are higher than the RL methods. The biggest advantage of our agent is mainly on the PnLMAP metrics, it can obtain exceeding profit while maintain a rather low inventory level. The AS strategy is also good at inventory controlling but at some expense of profitability.

E. Extended Experiments

In this section, three extended experiments on latency, ablation and explanation are conducted in *Ping An Bank Co.,Ltd.* to demonstrate the advantage of our method.

1) Latency: In HFT domain, latency, which is usually caused by messaging and calculation, is an important factor affecting the profitability of a strategy. This is because these quotes may not be based on the latest price information. Therefore, we are concerned about whether the agent can

TABLE III
THE RUNTIME OF METHODS.

Method	Random	Fixed	AS	D-DQN		C-PPO	
				Infer	Train	Infer	Train
Runtime (ms/ts)	10.0	10.9	19.7	46.7	77.5	47.5	75.7

TABLE IV
ABLATION EXPERIMENTS.

	ND-PnL ($\times 10^5$)	PnLMAP	Profit Ratio ($\times 10^{-4}$)	Sharpe
C-PPO	9.34	117.18	5.01	12.34
w/o LOB state	0.15	17.13	10.66	1.20
w/o Attn-LOB	0.58	22.18	11.15	1.43
w/o Dynamic state	9.12	112.52	5.05	13.32
D-DQN	6.98	8.65	3.54	1.25
w/o LOB state	4.52	6.60	1.70	2.71
w/o Attn-LOB	6.96	7.83	3.46	1.31
w/o Dynamic state	6.38	7.90	3.95	1.11

maintain its performance against latencies. We investigated the robustness of our approach and baselines to latency, as shown in Figure 2.

The AS, Random, and Fixed strategies suffer from latencies, which is because these models' parameters are fixed and therefore cannot accommodate latencies. And our RL agents, C-PPO and D-DQN, perform better than them, because the agent may self-adapt to latencies in the environment. Since our pre-training horizon is equal to 10, it performs better when the latency is around 10.

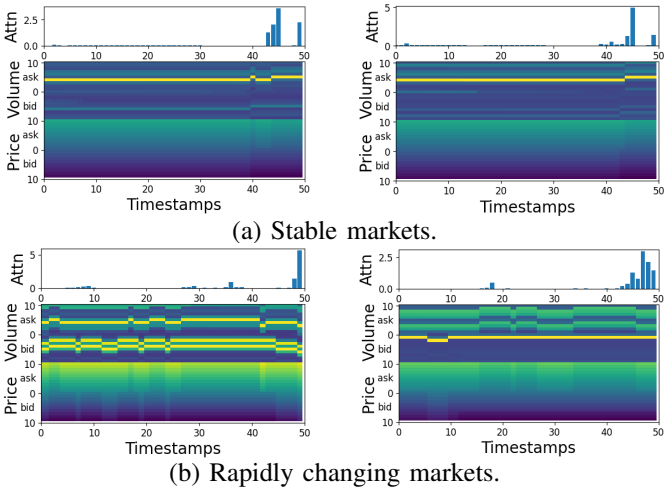
And we record the running time of different methods, as shown in Table III. Baselines have a huge advantage in running time, but considering the average interval between events is 60-150 ms, the computation time of our agents is acceptable.

2) Ablation: In order to investigate the effects of each part in our model, we perform ablation experiments, and the results are shown in Table IV.

The rich microstructure information contained in limit order books are key features to a market making agent. If we do not input the LOB state, the performance decreased severely. But it can only be extracted by a specially designed module, if we replace Attn-LOB with a simple MLP, the performance only increase slightly. The dynamic state, which represents the market dynamics, also enhanced the performance, but not so much.

3) Explanation: By visualizing the weight of self-attention layer in Figure 3, we find that the agent paid the most attention to the most recent events. In a stable market like (a), the agent will pay attention to the latest change, while in a rapidly changing market like (b), it will also look at the earlier market changes.

In Figure 4, we plot historical decisions of the C-PPO agent to show that it did learn some trading guidelines. In the first half of the picture, where the market fluctuated around a certain price, the agent made profits by opening and closing positions. When the agent holds a negative position, it puts the bid quote very near the mid-price in an effort to revert to a neutral position. However, in the second half of the picture, the



(a) Stable markets.

(b) Rapidly changing markets.

Fig. 3. Attention visualization.

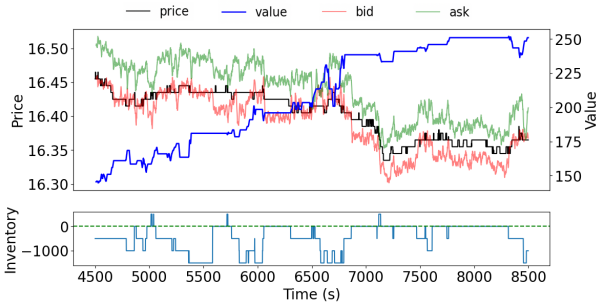


Fig. 4. An example of agent decision making.

price fell sharply and a trending market emerged. The quoted prices are far away from the mid-price, so as to keep a low inventory position to avoid risks.

V. CONCLUSION

In this paper, we propose a novel deep RL agent for market making, which shows the competitive ability for capturing spreads and controlling inventory, and the robustness to latencies. To achieve this, we leverage a pre-trained neural network Attn-LOB to extract features from limit order books. Then we propose a novel continuous action space and a hybrid reward function, and we show their effectiveness with ablation experiments. We explain our results by visualizing attention weights. The agent paid the most attention to the nearest events while still looking at very early changes. And we find that the agent can learn some market-making skills like humans. Empirically, the proposed agent achieved satisfactory performance, outperforming the traditional quantitative model and RL baselines on the simulated environment of three stocks.

REFERENCES

[1] H. R. Stoll, "Inferring the components of the bid-ask spread: Theory and empirical tests," *the Journal of Finance*, vol. 44, no. 1, pp. 115–134, 1989.

[2] I. Roşu, "A dynamic model of the limit order book," *The Review of Financial Studies*, vol. 22, no. 11, pp. 4601–4641, 2009.

[3] C. Cao, O. Hansch, and X. Wang, "The information content of an open limit-order book," *Journal of Futures Markets: Futures, Options, and Other Derivative Products*, vol. 29, no. 1, pp. 16–41, 2009.

[4] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quantitative Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.

[5] H. Demsetz, "The cost of transacting," *The quarterly journal of economics*, vol. 82, no. 1, pp. 33–53, 1968.

[6] M. B. Garman, "Market microstructure," *Journal of financial Economics*, vol. 3, no. 3, pp. 257–275, 1976.

[7] H. R. Stoll, "The pricing of security dealer services: An empirical study of nasdaq stocks," *The journal of finance*, vol. 33, no. 4, pp. 1153–1172, 1978.

[8] T. Ho and H. R. Stoll, "Optimal dealer pricing under transactions and return uncertainty," *Journal of Financial economics*, vol. 9, no. 1, pp. 47–73, 1981.

[9] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.

[10] T. Ho and H. R. Stoll, "On dealer markets under competition," *The Journal of Finance*, vol. 35, no. 2, pp. 259–267, 1980.

[11] F. Guilbaud and H. Pham, "Optimal high-frequency trading with limit and market orders," *Quantitative Finance*, vol. 13, no. 1, pp. 79–94, 2013.

[12] N. T. Chan and C. Shelton, "An electronic market-maker," 2001.

[13] A. Chakraborti, I. M. Toke, M. Patriarca, and F. Abergel, "Econophysics review: I. empirical facts," *Quantitative Finance*, vol. 11, no. 7, pp. 991–1012, 2011.

[14] A. S. Kyle, "Continuous auctions and insider trading," *Econometrica: Journal of the Econometric Society*, pp. 1315–1335, 1985.

[15] L. R. Glosten, "Is the electronic open limit order book inevitable?" *The Journal of Finance*, vol. 49, no. 4, pp. 1127–1161, 1994.

[16] M. O'hara, *Market microstructure theory*. John Wiley & Sons, 1998.

[17] J. Sirignano and R. Cont, "Universal features of price formation in financial markets: perspectives from deep learning," *Quantitative Finance*, vol. 19, no. 9, pp. 1449–1459, 2019.

[18] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Using deep learning for price prediction by exploiting stationary limit order book features," *Applied Soft Computing*, vol. 93, p. 106401, 2020.

[19] A. Ntakaris, G. Mirone, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Feature engineering for mid-price prediction with deep learning," *Ieee Access*, vol. 7, pp. 82 390–82 412, 2019.

[20] D. T. Tran, M. Magris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Tensor representation in high-frequency financial data for price change prediction," in *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2017, pp. 1–7.

[21] P. Nousi, A. Tsantekidis, N. Passalis, A. Ntakaris, J. Kanniainen, A. Tefas, M. Gabbouj, and A. Iosifidis, "Machine learning for forecasting mid-price movements using limit order book data," *Ieee Access*, vol. 7, pp. 64 722–64 736, 2019.

[22] Z. Zhang, S. Zohren, and S. Roberts, "Extending deep learning models for limit order books to quantile regression," *arXiv preprint arXiv:1906.04404*, 2019.

[23] ——, "Deeplob: Deep convolutional neural networks for limit order books," *IEEE Transactions on Signal Processing*, vol. 67, no. 11, pp. 3001–3012, 2019.

[24] Z. Zhang and S. Zohren, "Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units," *arXiv preprint arXiv:2105.10430*, 2021.

[25] A. Ntakaris, M. Magris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods," *Journal of Forecasting*, vol. 37, no. 8, pp. 852–866, 2018.

[26] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Forecasting stock prices from the limit order book using convolutional neural networks," in *2017 IEEE 19th conference on business informatics (CBI)*, vol. 1. IEEE, 2017, pp. 7–12.

[27] ——, "Using deep learning to detect price change indications in financial markets," in *2017 25th European Signal Processing Conference (EUSIPCO)*. IEEE, 2017, pp. 2511–2515.

[28] D. T. Tran, A. Iosifidis, J. Kanniainen, and M. Gabbouj, "Temporal attention-augmented bilinear network for financial time-series data analysis," *IEEE transactions on neural networks and learning systems*, vol. 30, no. 5, pp. 1407–1418, 2018.

[29] M. Shabani, D. T. Tran, M. Magris, J. Kanniainen, and A. Iosifidis, "Multi-head temporal attention-augmented bilinear network for financial time series prediction," *arXiv preprint arXiv:2201.05459*, 2022.

[30] J. Moody and M. Saffell, "Reinforcement learning for trading," *Advances in Neural Information Processing Systems*, vol. 11, 1998.

[31] ——, "Learning to trade via direct reinforcement," *IEEE transactions on neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.

[32] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE transactions on neural networks and learning systems*, vol. 28, no. 3, pp. 653–664, 2016.

[33] H. Wei, Y. Wang, L. Mangu, and K. Decker, "Model-based reinforcement learning for predictions and control for limit order books," *arXiv preprint arXiv:1910.03743*, 2019.

[34] A. Briola, J. Turiel, R. Marcaccioli, and T. Aste, "Deep reinforcement learning for active high frequency trading," *arXiv preprint arXiv:2101.07107*, 2021.

[35] Y. Nevmyvaka, Y. Feng, and M. Kearns, "Reinforcement learning for optimized trade execution," in *Proceedings of the 23rd international conference on Machine learning*, 2006, pp. 673–680.

[36] Z. Wang, B. Huang, S. Tu, K. Zhang, and L. Xu, "Deeptrader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 643–650.

[37] R. Wang, H. Wei, B. An, Z. Feng, and J. Yao, "Commission fee is not enough: A hierarchical reinforced framework for portfolio management," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 626–633.

[38] O. Jin and H. El-Saawy, "Portfolio management using reinforcement learning," *Stanford University*, 2016.

[39] J. Abernethy and S. Kale, "Adaptive market making via online learning," *Advances in Neural Information Processing Systems*, vol. 26, 2013.

[40] Y.-S. Lim and D. Gorse, "Reinforcement learning for high-frequency market making," in *ESANN 2018-Proceedings, European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*. ESANN, 2018, pp. 521–526.

[41] T. Spooner, J. Fearnley, R. Savani, and A. Koukorinis, "Market making via reinforcement learning," *arXiv preprint arXiv:1804.04216*, 2018.

[42] Y. Zhong, Y. Bergstrom, and A. R. Ward, "Data-driven market-making via model-free learning." in *IJCAI*, 2020, pp. 4461–4468.

[43] J. Sadighian, "Deep reinforcement learning in cryptocurrency market making," *arXiv preprint arXiv:1911.08647*, 2019.

[44] B. Gašperov and Z. Kostanjčar, "Market making with signals through deep reinforcement learning," *IEEE Access*, vol. 9, pp. 61 611–61 622, 2021.

[45] P. Kumar, "Deep reinforcement learning for market making," in *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, 2020, pp. 1892–1894.

[46] Z. Xu, X. Cheng, and Y. He, "Performance of deep reinforcement learning for high frequency market making on actual tick data," in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, 2022, pp. 1765–1767.

[47] M. Mani, S. Phelps, and S. Parsons, "Applications of reinforcement learning in automated market-making," in *Proceedings of the GAIW: Games, Agents and Incentives Workshops, Montreal, Canada*, 2019, pp. 13–14.

[48] J. W. Wilder, *New concepts in technical trading systems*. Trend Research, 1978.

[49] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[50] T. Chakraborty and M. Kearns, "Market making and mean reversion," in *Proceedings of the 12th ACM conference on Electronic commerce*, 2011, pp. 307–314.

[51] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "Wavenet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.